Microsoft Fabric & Power BI Governance & Administration

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Microsoft Fabric & Power BI Governance & Administration 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Boundary . . . 4

Effort . . . 5

1. Solution Overview — Microsoft Fabric & Power BI Governance & Administration 5

What this is . . . 5

How the technology works . . . 5

The standard deployment model . . . 6

Scope . . . 6

Out of scope . . . 7

Auth and connections used . . . 7

2. Requirements Guide — Microsoft Fabric & Power BI Governance & Administration 7

2.1 Admin portal access with the Fabric/Power BI Administrator role . . . 7

2.2 MicrosoftPowerBIMgmt PowerShell module . . . 8

2.3 Security groups for baseline scoping and workspace roles . . . 8

2.4 A capacity to manage and monitor . . . 8

2.5 Three workspaces (or the right to create them) for the lifecycle . . . 8

2.6 Service principal enabled for read-only admin APIs (for unattended scripts) . . . 9

2.7 A change-approval record for tenant-wide settings . . . 9

Global Requirements — Data (category-level) 9

G1. Microsoft 365 / Entra tenant with Fabric/Power BI enabled . . . 9

G2. Fabric / Power BI administrative role . . . 9

G3. Licensing and capacity . . . 10

G4. Identity, groups, and access model . . . 10

G5. Admin API access . . . 10

G6. Information protection alignment (where used) . . . 10

G7. Source control / lifecycle target . . . 10

G8. Change-control for tenant settings + a pilot ring . . . 10

3. Implementation Guide — Microsoft Fabric & Power BI Governance & Administration 11

Step 0 — Assess (inventory before you change anything) . . . 11

Step 1 — Configure the tenant-settings baseline (scoped to the pilot ring) . . . 11

Step 2 — Configure capacity management and workload settings . . . 12

Step 3 — Build the workspace architecture and access model . . . 13

Step 4 — Create the deployment pipeline (dev → test → prod lifecycle) . . . 13
Step 5 — Turn on admin monitoring . . . 14
Step 6 — Pilot and Validate . . . 14
Step 7 — Expand (tenant-wide, under change control) . . . 14
5. Testing & Validation — Microsoft Fabric & Power BI Governance & Administration 15
5.1 Baseline inventory captured (Assess gate) . . . 15
5.2 Tenant-settings baseline resolves for the ring . . . 15
5.3 Workspace roles match the access model . . . 15
5.4 Workspaces are on the correct capacity . . . 16
5.5 Deployment pipeline promotes content correctly . . . 16
5.6 Capacity utilization and activity monitoring work . . . 16
5.7 Tenant-wide expansion is safe (Expand gate) . . . 16
Sign-off . . . 17
6. Troubleshooting — Microsoft Fabric & Power BI Governance & Administration 17
Admin portal or scripts . . . 17
Tenant settings . . . 18
Capacity . . . 18
Deployment pipelines . . . 19
Access model . . . 19
7. Best Practices — Microsoft Fabric & Power BI Governance & Administration 19
Keep tenant settings clean . . . 19
Keep the access model clean . . . 20
Keep capacity healthy and costs visible . . . 20
Keep monitoring honest . . . 20
Defer the decisions that are not yours . . . 20
Where to go next (adjacent catalog areas) . . . 21
8. Deliverables Summary — Microsoft Fabric & Power BI Governance & Administration 21
Documentation (source guides) . . . 21
Scripts (PowerShell — read-only or preview-first) . . . 22
Naming convention . . . 22
Format . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Microsoft Fabric & Power BI Governance & Administration

Self-service analytics sprawls fast. Anyone can spin up a workspace, publish a report, share it with the whole company, and pin it to a capacity nobody is watching — so you end up with uncontrolled workspaces, over-shared and over-exported reports, mystery capacity spend, and no path from a draft to a trusted production report. This solution establishes governance at the **platform** level: a tenant- settings baseline, a capacity management and monitoring model, a workspace architecture with a role and access model, deployment pipelines for a controlled dev to test to prod lifecycle, and the admin and scanner APIs that let you see and prove what is happening.

This is a complete, do-it-the-repeatable-way package for governing Microsoft Fabric and Power BI as a platform, written for a competent Fabric/Power BI administrator with no prior tenant-governance experience. It is deliberately scoped to platform **administration and governance** — tenant settings, capacities, workspaces, pipelines, and monitoring.

What you get

- A plain-language Solution Overview of the platform-governance model, how the admin surface works, and the standard deployment model.
- A Requirements Guide with Verify and Implement-if-missing for every solution-specific prerequisite (category-wide prerequisites live in the separate global-requirements file).
- An Implementation Guide of precise, ordered, click-by-click steps — every object you create is named with one convention and reused through testing, troubleshooting, and the scripts.
- Five PowerShell scripts: platform discovery, a governance-baseline validator, a capacity-and-activity report, a tenant-settings desired-state helper, and a workspace/pipeline provisioner — read-only or preview-first by default.
- Testing and Troubleshooting guides so you can prove the governance actually holds before you expand.

What this delivers

A named tenant baseline — `Tenant Governance Baseline – Fabric/Power BI – v1` — applied to defined security groups, a workspace model (`Sales BI – Content – Dev/Test/Prod`) fed by a deployment pipeline (`Deployment Pipeline – Sales BI – DevTestProd`), capacities that are monitored for utilization and spend, and admin-API-based monitoring — all piloted on a governance ring before it touches the whole tenant.

The standard deployment model

Assess to Configure to Pilot to Validate to Expand — tenant-setting changes are piloted on a ring and approved before they go tenant-wide, because they affect every user.

Boundary

This solution governs the **platform**. Data classification of analytics content — sensitivity labeling and DLP for Power BI/Fabric, and Purview-side reporting — is delivered by the Microsoft Purview information-protection solutions and is cross-referenced here, not duplicated. Regulatory, records, and data-classification determinations remain the customer's compliance function's decision; this guide provides the technical controls.

Effort

Plan roughly two to four days for a first rollout: about half a day to assess and inventory, a day to configure the baseline, workspaces, and a pilot pipeline, a day to pilot and validate on the governance ring, and the remainder to expand tenant-wide under change control. Reusable on every subsequent workload and rollout.

1. Solution Overview — Microsoft Fabric & Power BI Governance & Administration

What this is

A platform-governance solution for Microsoft Fabric and Power BI. It puts controls around the shared service that self-service analytics runs on, so growth stays deliberate instead of chaotic. You will set a tenant-settings baseline, build a workspace and access model, put content on a controlled dev to test to prod lifecycle with deployment pipelines, manage and monitor the capacities that content runs on, and turn on admin-API-based monitoring so you can see and prove what is happening.

This is platform **administration and governance**. It is not report authoring, not data modeling, and not data classification. Where classification is in scope — sensitivity labels and DLP for Power BI / Fabric, and Purview-side reporting — that is delivered by the Microsoft Purview information-protection solutions and cross-referenced here.

How the technology works

Microsoft Fabric and Power BI share one administrative fabric on top of your Microsoft 365 / Entra tenant. The pieces you will govern:

- **Tenant settings** — a single admin surface (the Fabric/Power BI admin portal) that turns

capabilities on or off for the whole tenant or for named security groups: who can share externally, who can export data, who can publish to web, who can certify content, who can create workspaces, and which service principals may call the admin APIs. Tenant settings apply to **every user**, so they are the highest-leverage and highest-risk control — always scoped to groups and piloted first.

- **Capacities** — the compute that Fabric and Power BI Premium content runs on (Fabric F-SKUs or Power

BI Premium P-SKUs). Capacities carry workload settings and are the unit of cost. Left unmonitored they either throttle (overloaded) or waste money (idle). *Verify the current SKU-to-feature mapping in your tenant.*

- **Workspaces** — the containers for content (reports, semantic models, lakehouses, pipelines). Each

has roles — Admin, Member, Contributor, Viewer — that you assign to security groups, not people. Workspaces are assigned to a capacity, which is where governance meets cost.

- **Deployment pipelines** — a built-in dev to test to prod promotion mechanism. You pair three

workspaces to the three stages and promote content forward with deployment rules, so production is never edited directly.

- **Admin and scanner APIs / activity log** — the read surface. The admin REST APIs and the metadata

scanner return a tenant-wide inventory of workspaces, artifacts, and permissions; the activity log (and the admin monitoring / metrics experiences) return who did what. This is how you audit sharing, find orphaned or over-shared content, and watch capacity utilization.

The standard deployment model

Every solution in this catalog follows the same five-step shape so repeat rollouts feel familiar:

1. **Assess** — inventory current tenant settings, capacities, workspaces, and sharing with the

discovery script; identify sprawl and risky settings; agree the target baseline with the business.

1. **Configure** — author the tenant-settings baseline scoped to groups, build the workspace model and

access groups, create the deployment pipeline, and set capacity workload/monitoring — all against a pilot ring first, nothing tenant-wide yet.

1. **Pilot** — apply the baseline and workspace/pipeline model to the governance pilot ring only; run a

real content promotion dev to test to prod; watch capacity behavior.

1. **Validate** — run the validation script and the manual checks; confirm each control behaves as

intended and nothing legitimate broke; get change approval for tenant-wide application.

1. **Expand** — roll the approved baseline tenant-wide under change control, onboard additional

workloads onto the workspace/pipeline pattern, and put the monitoring report on a schedule.

Scope

- A tenant-settings governance baseline (sharing, export, publish-to-web, certification, workspace

creation, service-principal admin-API access), scoped to security groups.

- Capacity management: workload settings, assignment of workspaces to capacities, and utilization / cost

monitoring.

- A workspace architecture and access model with roles assigned to groups.

- Deployment pipelines (dev to test to prod) for a controlled content lifecycle.

- Admin monitoring via the activity log, the admin monitoring / metrics experiences, and the admin /

scanner REST APIs.

Out of scope

- **Report and semantic-model authoring / data modeling** — this is platform governance, not content creation.
- **Sensitivity labeling and DLP for Power BI / Fabric, and Purview-side reporting** — delivered by the Microsoft Purview information-protection solutions; classification determinations are the compliance team's decision.
- **Row-level / object-level security design inside a semantic model** — a content concern, not a platform one.
- **Regulatory, records-retention, and legal determinations** — the customer's compliance/legal function decides; this guide provides the technical controls.

Auth and connections used

- The **Fabric/Power BI admin portal** (`https://app.powerbi.com` → Settings → Admin portal) for tenant settings, capacities, and monitoring — needs the Fabric Administrator / Power BI Administrator role (some settings need Global Administrator).
- **Microsoft Entra security groups** to scope every tenant setting and every workspace role.
- The **MicrosoftPowerBIMgmt** PowerShell module and the Fabric/Power BI **admin & scanner REST APIs**, called either as a delegated admin or via a service principal that the "service principals can use read-only admin APIs" tenant setting permits, for discovery, validation, and reporting.

2. Requirements Guide — Microsoft Fabric & Power BI Governance & Administration

Verify every requirement before you configure anything. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites (tenant with Fabric/Power BI enabled, admin role, licensing/capacity, identity/groups, admin-API access, Purview alignment, lifecycle target, change control + pilot ring) are in `global-requirements-data.md` — do not repeat them; this file covers only what is specific to this solution.

2.1 Admin portal access with the Fabric/Power BI Administrator role

Description: The operator can open the Fabric/Power BI admin portal and edit tenant settings. Why: The entire tenant-settings baseline, capacity settings, and monitoring live behind the admin portal, which requires the admin role (global requirement G2). Verify: Go to `https://app.powerbi.com` → Settings (gear) → **Admin portal**; the **Tenant settings**, **Capacity settings**, and **Monitoring** pages are visible and editable. Implement: Assign the Fabric Administrator (or Power BI Administrator) role in Entra → Roles and

administrators. Reserve Global Administrator for the few settings that require it.

2.2 MicrosoftPowerBIMgmt PowerShell module

Description: The MicrosoftPowerBIMgmt module on the workstation running the scripts. Why: The discovery, validation, report, tenant-settings, and workspace-provision scripts call the Power BI / Fabric admin REST APIs through this module (`Connect-PowerBIServiceAccount`, `Invoke-PowerBIRestMethod`, `New-PowerBIWorkspace`). Verify: `Get-Module -ListAvailable MicrosoftPowerBIMgmt` returns a version. Implement: Run `Install-Module MicrosoftPowerBIMgmt -Scope CurrentUser`.

2.3 Security groups for baseline scoping and workspace roles

Description: Entra security groups that will (a) scope tenant settings and (b) receive workspace roles, plus the governance pilot ring. Why: Governance is applied to groups, not individuals (global G4), so tenant settings and workspace access stay manageable and auditable, and the pilot ring lets you test tenant-scoped settings safely. Verify: The intended groups exist in Entra → Groups. Run `FABGOV-Discovery.ps1`, which lists the groups it expects. Implement: Create these security groups before configuring (names used throughout this guide):

- `Governance Pilot Ring – Fabric – Ring1` (the pilot audience for tenant-scoped settings)
- `Fabric Admins – Platform – Prod` (holders of the admin role / admin-portal access)
- `BI Creators – Analytics – Pilot` (users allowed to create workspaces during pilot)
- `BI Content Certifiers – Analytics – Prod` (users allowed to certify content)
- `Fabric Admin API Apps – Platform – Prod` (service principals permitted to call read-only admin APIs)

2.4 A capacity to manage and monitor

Description: At least one Fabric (F-SKU) or Power BI Premium (P-SKU) capacity in the tenant. Why: Capacity management, workload settings, deployment pipelines' production stage, and utilization/cost monitoring all require a capacity (global G3). *Verify the current SKU-to-feature mapping in your tenant — feature gating changes.* Verify: Admin portal → **Capacity settings**; at least one capacity is listed and Active. Run `FABGOV-Discovery.ps1` to confirm it is returned by the admin API. Implement: Acquire and provision a Fabric F-SKU (or Power BI Premium P-SKU) capacity, and record its name as `Fabric Capacity – Analytics – Prod`.

2.5 Three workspaces (or the right to create them) for the lifecycle

Description: Distinct dev, test, and prod workspaces for at least one pilot workload. Why: Deployment pipelines pair one workspace to each of the three stages; production must never be edited directly (global G7). Verify: In the Power BI service you can create workspaces and assign at least one to the capacity. Implement: The implementation guide creates `Sales BI – Content – Dev`, `Sales BI – Content – Test`, and `Sales BI – Content – Prod`. Confirm you have rights to create them (governed by the "create workspaces" tenant setting).

2.6 Service principal enabled for read-only admin APIs (for unattended scripts)

Description: An Entra app registration whose service principal is permitted to call the read-only admin APIs, if you want the discovery/report scripts to run unattended. Why: Scheduled monitoring (the report script on a schedule) should not depend on a person's interactive sign-in; the scanner and admin read APIs support a service principal when the tenant setting allows it (global G5). Verify: Admin portal → Tenant settings → **Admin API settings** → "Service principals can access read-only admin APIs" is enabled for `Fabric Admin API Apps – Platform – Prod`, and the app is a member of that group. *Verify the exact setting name in your tenant; the admin-API settings surface changes.* Implement: Register an Entra app, add its service principal to `Fabric Admin API Apps – Platform – Prod`, and enable the tenant setting for that group. Interactive (delegated) runs do not need this.

2.7 A change-approval record for tenant-wide settings

Description: A documented approval step and rollback note for each tenant-setting change (global G8). Why: Tenant settings affect every user; the pilot proves the setting, but tenant-wide application must be approved and reversible. Business decisions about which settings to enforce are the customer's. Verify: A change ticket / record template exists that captures the setting, the before value, the after value, the approver, and the rollback. Implement: Adopt the change-control record before the Expand step; the implementation guide references it at each tenant-wide apply.

Global Requirements — Data (category-level)

These requirements apply to every solution in the Data category (Microsoft Fabric / Power BI and related analytics-platform administration and governance). Published once at the category level so listings stay small. Confirm before deploying any Data solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Analytics platforms sprawl through self-service. Govern at the platform level (tenant settings, capacities, workspaces) before locking down individual content, change tenant settings in a controlled way (they affect every user), and pilot governance in a ring before enforcing tenant-wide.*

G1. Microsoft 365 / Entra tenant with Fabric/Power BI enabled

Description: A tenant with Microsoft Fabric / Power BI service enabled. Why: All administration and governance operate against the tenant's Fabric/Power BI service. Verify: Sign in to `https://app.powerbi.com` / the Fabric portal; the admin portal is reachable. Implement: Enable Fabric/Power BI and confirm the service is provisioned.

G2. Fabric / Power BI administrative role

Description: Fabric Administrator / Power BI Administrator (and Global Administrator for some tenant settings). Why: Tenant settings, capacity management, and the admin APIs require an admin role. Verify: Entra -> `Roles and administrators`; confirm Fabric Administrator (or Power BI Administrator). Implement: Assign the Fabric/Power BI Administrator role; reserve Global Administrator for tenant-wide changes.

G3. Licensing and capacity

Description: The Fabric/Power BI licensing in scope — Power BI Pro/PPU per user and/or Fabric (F-SKU) or Power BI Premium (P-SKU) capacity. Why: Capacity management, deployment pipelines, and some governance features are capacity/license-gated. Verify: Admin portal -> capacity settings and `Microsoft 365 admin center -> Billing -> Licenses`. *Verify current SKU/feature mapping in your tenant.* Implement: Acquire/assign the required licenses and capacity before configuring capacity-gated features.

G4. Identity, groups, and access model

Description: Entra ID security groups to drive workspace roles and tenant-setting scoping. Why: Governance is applied to groups, not individuals, so access stays manageable and auditable. Verify: The intended Entra groups exist. Run the solution's discovery script. Implement: Create the security groups (see the Entra category solutions) before assigning workspace/access roles.

G5. Admin API access

Description: Access to the Fabric/Power BI admin and scanner REST APIs (via delegated admin or a service principal enabled for admin APIs). Why: Inventory, activity logs, and metadata scanning are collected through the admin APIs. Verify: The admin API is reachable with the operator's token / service principal. *Verify service-principal-for-admin-API tenant setting.* Implement: Enable the "service principals can use read-only admin APIs" tenant setting and grant the app the needed scope.

G6. Information protection alignment (where used)

Description: Microsoft Purview sensitivity labels available to Fabric/Power BI where labeling/DLP is in scope. Why: Data-classification of analytics content is delivered via Purview, not the Fabric admin surface. Verify: Sensitivity labels are published and enabled for Fabric/Power BI. Implement: Publish labels (see the Purview information-protection solutions). *Data-classification determinations are the compliance team's decision; this guide covers platform administration.*

G7. Source control / lifecycle target

Description: A dev/test/prod separation (workspaces and, where used, Git integration) for content lifecycle. Why: Deployment pipelines and safe promotion need distinct stages. Verify: Separate dev/test/prod workspaces (or a Git-integrated workspace) exist or can be created. Implement: Create the staged workspaces and connect Git integration where the solution uses it.

G8. Change-control for tenant settings + a pilot ring

Description: A change-approval process for tenant-setting changes and a pilot group to validate against. Why: Tenant settings affect every user; changes are piloted on a ring and approved before tenant-wide. Verify: A pilot group exists; tenant-setting changes follow an approval step. Implement: Define the pilot ring and approval step before applying tenant-wide governance. *Tenant-setting business decisions are the customer's.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Microsoft Fabric & Power BI Governance & Administration

Follow these steps in order. Every object you create is named with one convention — `<Purpose> – <Scope> – <Stage>` — and reused by that exact name in later steps, in testing, in troubleshooting, and in the scripts. Do everything against the **pilot ring first**; nothing goes tenant-wide until the Validate step and a change approval.

Portal paths are current at time of writing; Microsoft moves menus, so if a path differs, search the admin portal for the setting name. Sign in throughout at `https://app.powerbi.com` with an account that holds the Fabric Administrator / Power BI Administrator role.

Objects created in this guide (reuse these exact names):

- Groups: `Governance Pilot Ring – Fabric – Ring1`, `Fabric Admins – Platform – Prod`, `BI Creators – Analytics – Pilot`, `BI Content Certifiers – Analytics – Prod`, `Fabric Admin API Apps – Platform – Prod`
- Baseline: `Tenant Governance Baseline – Fabric/Power BI – v1`
- Capacity: `Fabric Capacity – Analytics – Prod`
- Workspaces: `Sales BI – Content – Dev`, `Sales BI – Content – Test`, `Sales BI – Content – Prod`
- Pipeline: `Deployment Pipeline – Sales BI – DevTestProd`

Step 0 — Assess (inventory before you change anything)

1. On the workstation, open PowerShell and run the discovery script:

```
powershell -ExecutionPolicy Bypass -File .\FABGOV-Discovery.ps1
```

1. Sign in when prompted (delegated admin). The script prints current **tenant settings** of interest,

the list of **capacities**, a **workspace inventory** (with those that are personal or orphaned flagged), and a **sharing/export** summary from the activity log.

1. Save the output. This is your "before" picture and the evidence you will compare against in Validate.
2. Create the five security groups from requirement 2.3 in Entra → Groups if they do not exist. Add a

handful of willing users to `Governance Pilot Ring – Fabric – Ring1` — this is your safe test audience for tenant-scoped settings.

Step 1 — Configure the tenant-settings baseline (scoped to the pilot ring)

You will author `Tenant Governance Baseline – Fabric/Power BI – v1`. For **every** setting below, in the admin portal, set it to **apply to specific security groups** and choose `Governance Pilot Ring – Fabric – Ring1` (except where noted), so you are testing on the ring, not the tenant. Do not select "The entire organization" yet.

1. Go to Settings (gear) → **Admin portal** → **Tenant settings**.

2. **Export and sharing settings** — configure and pilot-scope each:

• **Publish to web** → set to **disabled** (or "Only existing embed codes"); publish-to-web exposes reports anonymously and is the highest-risk export.

• **Export to Excel / Export data** → enable only for `Governance Pilot Ring – Fabric – Ring1` during pilot.

• **Share content with external users** / **Guest users can access** → enable only for a named group, not the whole org.

• **Allow users to apply sensitivity labels** — leave alignment to the Purview solution; note it here but defer classification decisions to compliance.

1. **Workspace settings**:

• **Create workspaces (new experience)** → apply to **specific security groups** → `BI Creators – Analytics – Pilot`. This stops anyone from spinning up a workspace.

1. **Content certification** (Export and sharing / Discovery):

• **Certification** → enable, apply to `BI Content Certifiers – Analytics – Prod`, and set the certification support URL to your governance page. Only certifiers can mark content "Certified".

1. **Admin API settings**:

• **Service principals can access read-only admin APIs** → enable for `Fabric Admin API Apps – Platform – Prod` (needed by 2.6). *Verify the exact setting name; the admin-API surface changes.*

• **Enhance admin API responses with detailed metadata / DAX and mashup expressions** → enable so the scanner returns lineage detail for monitoring.

1. Record every change (setting, before value, after value) in your change-control record. Do **not** flip anything to "entire organization" in this step.

> *Any setting that is in preview or that gates on a specific license (for example some Fabric workload or Copilot-in-Fabric settings) is marked as such in the portal — *verify current availability and licensing in your tenant before relying on it.**

Step 2 — Configure capacity management and workload settings

1. Admin portal → **Capacity settings** → open `Fabric Capacity – Analytics – Prod`.

2. Under **Workloads / capacity settings**, review the workload limits (for example semantic-model memory, max intermediate row set) and set them to your agreed values. Leave defaults unless you have a measured reason to change them.

1. Under **Contributor permissions / capacity admins**, set the capacity admins to `Fabric Admins – Platform – Prod` and restrict who can assign workspaces to this capacity.

1. Turn on capacity monitoring: install the **Microsoft Fabric Capacity Metrics** app from `https://app.powerbi.com` → Apps → Get apps → search "Fabric Capacity Metrics", and connect it to `Fabric Capacity – Analytics – Prod`. This is your utilization and throttling view.

1. Note the capacity's SKU and region for the cost baseline; the `FABGOV-Report.ps1` script will pull utilization to compare against it.

Step 3 — Build the workspace architecture and access model

1. In the Power BI service, click **Workspaces** → **New workspace**. Create three workspaces: `Sales BI – Content – Dev`, `Sales BI – Content – Test`, `Sales BI – Content – Prod`.

1. For each workspace, open **Workspace settings** → **Premium/License** and assign it to `Fabric Capacity – Analytics – Prod` (Prod and Test at minimum; Dev may stay on shared/Pro during pilot).

1. Assign **roles to groups, never to individuals** (open each workspace → **Manage access → Add people or groups**):

   • `Sales BI – Content – Dev` → `BI Creators – Analytics – Pilot` = **Contributor**; `Fabric Admins – Platform – Prod` = **Admin**.

   • `Sales BI – Content – Test` → `BI Content Certifiers – Analytics – Prod` = **Member**; `Fabric Admins – Platform – Prod` = **Admin**.

   • `Sales BI – Content – Prod` → viewers get access via an **app** or a Viewer group, not direct edit; `Fabric Admins – Platform – Prod` = **Admin** only. Keep Prod edit rights minimal.

1. You now have a role model where creation is gated, editing is delegated to Dev, and Prod is promotion-only.

Step 4 — Create the deployment pipeline (dev → test → prod lifecycle)

1. In the service, go to **Deployment pipelines** (left nav) → **Create pipeline**. Name it `Deployment Pipeline – Sales BI – DevTestProd`. Choose three stages (Development, Test, Production).

1. **Assign a workspace to each stage**: Development → `Sales BI – Content – Dev`, Test → `Sales BI – Content – Test`, Production → `Sales BI – Content – Prod`.

1. Put a piece of pilot content (a simple report + semantic model) in `Sales BI – Content – Dev`.

2. Click **Deploy to Test**, then later **Deploy to Production**, to prove the promotion path. Use the **compare** view to confirm only intended changes move forward.

1. Set **deployment rules** on the Production stage (for example, point the semantic model at the production data source) so promotion swaps connections automatically instead of by hand.

1. Manage pipeline access so only `Fabric Admins – Platform – Prod` can deploy to Production — promotion to prod is a governed action.

Step 5 — Turn on admin monitoring

1. Admin portal → **Monitoring / Audit logs**: confirm the **activity log** is being retained and, if your organization streams it, that it flows to your Microsoft 365 / Purview audit pipeline. (Purview- side reporting is delivered by the Purview solutions; here you only confirm the log is available.)

1. Admin portal → open the **admin monitoring** / **Feature usage and adoption** report if available in your tenant, and the **Fabric Capacity Metrics** app from Step 2 — these are your standing dashboards.

1. Confirm the **scanner API** is reachable by running `FABGOV-Discovery.ps1` again; it should now return the enhanced metadata you enabled in Step 1.5.

1. Schedule `FABGOV-Report.ps1` (see the testing guide) to produce the weekly capacity-and-activity summary, using the service principal from requirement 2.6 for unattended runs.

Step 6 — Pilot and Validate

1. With everything scoped to `Governance Pilot Ring – Fabric – Ring1`, have the ring users work normally for a short pilot window: create content in Dev, promote through the pipeline, share, and export.

1. Run `FABGOV-Validate.ps1`. It checks that the baseline settings resolve as intended for the ring, that workspace roles match the model, that the workspaces are on the right capacity, and that the pipeline stages are paired correctly. See the testing guide for the pass/fail gates.

1. Confirm nothing legitimate broke (a ring user can still do their real work; only the risky actions are constrained).

1. Record results and obtain change approval (global G8) before any tenant-wide application.

Step 7 — Expand (tenant-wide, under change control)

1. For each approved tenant setting, return to Admin portal → Tenant settings and change the scope from

`Governance Pilot Ring – Fabric – Ring1` to the approved production scope (a broad "all except" group, or the entire organization where the business approved it). Change **one setting at a time** and re-run `FABGOV-Validate.ps1` after each.

1. Onboard additional workloads by repeating Steps 3–4 with new `<Workload> – Content – Dev/Test/Prod`

workspaces and their own pipeline, reusing the same groups and capacity model.

1. Keep `FABGOV-Report.ps1` on its schedule and review capacity utilization and sharing/export trends at

your governance cadence. Any regression or unexpected spend is your signal to revisit a setting under change control (with the recorded rollback).

5. Testing & Validation — Microsoft Fabric & Power BI Governance & Administration

Validate on the pilot ring before you expand tenant-wide. Each test states what to run, what passes, and what fails. The scripts referenced here are in the `scripts/` folder of this package.

5.1 Baseline inventory captured (Assess gate)

- Run: `FABGOV-Discovery.ps1`.
- Pass: the script returns the current tenant settings of interest, at least the capacity

`Fabric Capacity – Analytics – Prod`, a workspace inventory, and a sharing/export summary, with no authentication errors.

- Fail: an admin-permission or module error — fix the admin role (2.1) or install MicrosoftPowerBIMgmt

(2.2) before continuing. You cannot validate change without a "before" picture.

5.2 Tenant-settings baseline resolves for the ring

- Run: `FABGOV-Validate.ps1` after Step 1, then manually spot-check in the admin portal.
- Pass: for a member of `Governance Pilot Ring – Fabric – Ring1`, Publish to web is disabled, export is

scoped as configured, workspace creation is limited to `BI Creators – Analytics – Pilot`, and certification is limited to `BI Content Certifiers – Analytics – Prod`. The validator reports each setting as **Matches baseline**.

- Fail: any setting still shows "entire organization" when the baseline expects the ring, or a setting is

scoped to the wrong group. Re-open the setting and re-apply the group scope.

- Manual proof: sign in as (or ask) a ring user to attempt Publish to web — the option should be absent

or blocked.

5.3 Workspace roles match the access model

• Run: `FABGOV-Validate.ps1` (workspace-roles section).

• Pass: `Sales BI – Content – Dev/Test/Prod` each show only group-based role assignments matching Step 3

(no individual users; Prod has no broad edit rights). The validator reports **Roles match model**.

• Fail: an individual user holds a role, or a workspace has an unexpected Admin/Member — remove it and

re-assign to the correct group.

5.4 Workspaces are on the correct capacity

• Run: `FABGOV-Validate.ps1` (capacity-assignment section) and confirm in the Fabric Capacity Metrics

app.

• Pass: `Sales BI – Content – Test` and `Sales BI – Content – Prod` are assigned to `Fabric Capacity – Analytics – Prod`; the metrics app shows them consuming from it.

• Fail: a workspace is unassigned or on the wrong capacity — reassign in Workspace settings → Premium/License.

5.5 Deployment pipeline promotes content correctly

• Run: a manual promotion in `Deployment Pipeline – Sales BI – DevTestProd`.

• Pass: content created in `Sales BI – Content – Dev` deploys to Test and then Production; the compare

view shows only intended differences; deployment rules swap the production data source; only `Fabric Admins – Platform – Prod` can deploy to Production.

• Fail: promotion errors, an unexpected diff, or a non-admin can deploy to Prod — re-check stage-to-workspace pairing (Step 4.2), deployment rules (4.5), and pipeline access (4.6).

5.6 Capacity utilization and activity monitoring work

• Run: `FABGOV-Report.ps1`.

• Pass: the report returns capacity utilization for `Fabric Capacity – Analytics – Prod` and an activity

summary (top sharers, exports, publish-to-web attempts) from the activity log without error, and the numbers reconcile with the Fabric Capacity Metrics app.

• Fail: empty or errored output — confirm admin-API access (2.6) and that Step 5 enabled the log and enhanced metadata.

5.7 Tenant-wide expansion is safe (Expand gate)

• Run: `FABGOV-Validate.ps1` after each single tenant-wide setting change in Step 7.

• Pass: the changed setting reports **Matches baseline** at the new production scope, and no previously passing check regressed.

• Fail: any regression — roll the setting back using the recorded rollback value and re-pilot before retrying. Never batch multiple tenant-wide changes between validations.

Sign-off

Record for each test: date, operator, script output location, pass/fail, and (for tenant-wide changes) the change-approval reference. A clean run of 5.1–5.6 on the ring plus an approved 5.7 per setting is the bar for declaring the governance baseline in production.

6. Troubleshooting — Microsoft Fabric & Power BI Governance & Administration

Common failures, their cause, and the fix. Symptom → Cause → Fix.

Admin portal or scripts

The Admin portal / Tenant settings page is not visible

• Cause: the signed-in account lacks the Fabric Administrator / Power BI Administrator role (requirement 2.1 / global G2).

• Fix: assign the role in Entra → Roles and administrators, sign out and back in. A few settings additionally require Global Administrator — use a global admin for those.

`Connect-PowerBIServiceAccount` *or* `Invoke-PowerBIRestMethod` *fails*

• Cause: MicrosoftPowerBIMgmt not installed, or the account is not an admin, or you called an `admin/...` endpoint without admin rights.

• Fix: `Install-Module MicrosoftPowerBIMgmt -Scope CurrentUser`; confirm the admin role; ensure the URL uses the `admin/` prefix only when signed in as an admin.

Scripts return empty capacities/workspaces but no error

• Cause: the token is valid but the account has no admin scope, so the admin API returns an empty set.

• Fix: re-authenticate as a Fabric/Power BI Administrator; for unattended runs, confirm the service principal is in `Fabric Admin API Apps – Platform – Prod` and the "service principals can use read-only admin APIs" tenant setting is enabled (2.6 / global G5).

Activity-log calls return nothing

• Cause: the requested window exceeds the API limit (activity events are queried per day, up to ~30 days back), the log is not yet populated, or auditing is not enabled.

• Fix: keep `-Days` at 30 or fewer; confirm auditing is on for the tenant; allow time for events to appear (there is ingestion latency).

`FABGOV-Report.ps1 -ServicePrincipal` *fails to connect*

- Cause: missing `-TenantId/-AppId/-AppSecret`, an expired secret, or the app is not permitted for admin APIs.
- Fix: supply all three parameters from a secret store; rotate the secret if expired; confirm the app's service principal is enabled for read-only admin APIs.

Tenant settings

A tenant-setting change did not take effect for a user

- Cause: the setting is scoped to a security group the user is not in, or Entra group membership has not propagated yet.
- Fix: confirm the user's group membership (`Governance Pilot Ring – Fabric – Ring1` during pilot); allow propagation time; re-check with `FABGOV-Validate.ps1`.

Legitimate work broke after applying the baseline

- Cause: a control (export, external sharing, workspace creation) was scoped too tightly, catching real users.
- Fix: because you piloted on the ring, only ring users are affected — widen the enabled group or add the user; this is exactly the signal the pilot exists to surface before tenant-wide expansion.

A setting only offers "entire organization" (no group scope)

- Cause: some tenant settings are not group-scopable (`canSpecifySecurityGroups` is false).
- Fix: treat it as an all-or-nothing decision, get explicit change approval, and record the rollback; the baseline script flags these as "org-wide only".

Publish to web is still available to a pilot user

- Cause: the setting was saved to the wrong group, or an existing embed code predates the change.
- Fix: re-scope Publish to web to disabled/limited for the ring; review and delete existing embed codes in Admin portal → Embed codes.

Capacity

Reports are slow or throttled

- Cause: the capacity (`Fabric Capacity – Analytics – Prod`) is overloaded — too many/too-heavy workloads for its SKU.
- Fix: open the Fabric Capacity Metrics app, identify the top consumers, move non-production workloads off the capacity, or resize the SKU. *Verify current SKU sizing guidance for your workload.*

A workspace will not assign to the capacity

- Cause: the operator lacks capacity-assignment permission, or the workspace license mode is wrong.
- Fix: ensure `Fabric Admins – Platform – Prod` holds capacity-admin/assignment rights; set the workspace to the correct Premium/Fabric license mode in Workspace settings before assigning.

Deployment pipelines

"Deploy to Test/Production" is greyed out or errors

• Cause: a stage is not paired to a workspace, the target stage needs a capacity, or the operator lacks pipeline access.

• Fix: confirm each stage is paired (Dev/Test/Prod → the matching `Sales BI – Content – *`), ensure Test/Prod workspaces are on `Fabric Capacity – Analytics – Prod`, and that only `Fabric Admins – Platform – Prod` (with pipeline access) deploys to Prod.

Promotion moved more than expected

• Cause: no deployment rules, so data-source/connection differences promoted along with content.

• Fix: set deployment rules on the Production stage to swap data sources; use the compare view before every deploy.

Access model

`FABGOV-Validate.ps1` *flags "individual user holds roles"*

• Cause: a workspace role was assigned to a person instead of a group.

• Fix: remove the user in Manage access and add the correct group (`BI Creators – Analytics – Pilot`, `BI Content Certifiers – Analytics – Prod`, or `Fabric Admins – Platform – Prod`); re-run the validator.

`Add-PowerBIWorkspaceUser` *fails with a group name*

• Cause: the cmdlet may require the group's Entra object ID rather than its display name.

• Fix: resolve the group's object ID (Entra → Groups) and pass it as the `-Identifier`; the provision script emits this hint on failure.

7. Best Practices — Microsoft Fabric & Power BI Governance & Administration

Guidance to keep the platform governed after go-live, and where to go next in the catalog.

Keep tenant settings clean

• **Scope to groups, never the whole org by reflex.** "Enabled for the entire organization" is the

setting most likely to cause a later incident. Prefer "enabled for specific security groups" or "enabled for the entire organization except" and manage membership, not the setting.

• **Pilot every disruptive change on** `Governance Pilot Ring – Fabric – Ring1` **first**, then expand under

change control with a recorded before-value and rollback. Tenant settings affect every user.

• **Change one setting at a time** and re-validate. Batched tenant-wide changes make regressions

impossible to attribute.

• **Review the risky exports quarterly** — Publish to web, Export to Excel/CSV, external sharing, guest access. These are where data leaves the platform.

Keep the access model clean

• **Roles go to groups.** If you ever assign a workspace role to a person, treat it as debt to clean up.

• **Minimize Prod edit rights.** Production content changes through `Deployment Pipeline – Sales BI – DevTestProd, not by editing Sales BI – Content – Prod` directly.

• **Gate workspace creation** (`BI Creators – Analytics – Pilot` / your production creator group) so the workspace list stays intentional, and reclaim orphaned or personal-turned-shared workspaces the discovery script flags.

• **Use certification deliberately** — only `BI Content Certifiers – Analytics – Prod` marks content Certified, and only after it is genuinely production-quality, so the endorsement means something.

Keep capacity healthy and costs visible

• **Watch the Fabric Capacity Metrics app** for throttling and overload on `Fabric Capacity – Analytics – Prod`; size the capacity to the measured peak, not a guess.

• **Run** `FABGOV-Report.ps1` **on a schedule** and review utilization and sharing/export trends at a fixed governance cadence. Unexplained spend or a spike in exports is a signal, not noise.

• **Assign workspaces to capacity on purpose** — keep dev/experimental content off the production capacity where it can starve real workloads.

Keep monitoring honest

• **Prefer read-only discovery.** The discovery, validate, and report scripts are read-only by default; keep them that way and run change scripts (tenant settings, provisioning) preview-first.

• **Run unattended monitoring as a service principal** (`Fabric Admin API Apps – Platform – Prod`), not a person's account, so reporting survives staff changes.

• **Keep the change-control record** of every tenant-setting change; it is your audit trail and your rollback map.

Defer the decisions that are not yours

• **Data classification** — which content is confidential, and the labeling/DLP policy — is the

compliance team's decision, delivered through Microsoft Purview, not the Fabric admin surface.

- **Regulatory, records-retention, and legal determinations** belong to compliance/legal; this guide provides platform controls, not the policy.
- **Which capabilities to allow** (external sharing, publish to web) is a business risk decision; you implement and pilot it, the business owns it.

Where to go next (adjacent catalog areas)

- **Microsoft Purview — information protection for Power BI/Fabric** (sensitivity labels + DLP) and **Purview reporting** — for the classification layer this solution deliberately does not cover.
- **Microsoft Entra identity & groups solutions** — to industrialize the security-group model that every tenant setting and workspace role depends on.
- **Data-platform reporting solutions** (for example license usage & cost dashboards in Power BI) — once the platform is governed, build governed content on top of it.
- **Azure monitoring / cost-management solutions** — to fold Fabric capacity spend into broader FinOps reporting.

8. Deliverables Summary — Microsoft Fabric & Power BI Governance & Administration

Everything included in this solution package.

Documentation (source guides)

- `executive-summary.md` — what the solution is, the outcome, the boundary, and the effort.
- `01-solution-overview.md` — what it is, how the Fabric/Power BI admin surface works, the standard deployment model (Assess → Configure → Pilot → Validate → Expand), scope, out-of-scope, and the auth/connections used.
- `02-requirements.md` — solution-specific prerequisites (Verify / Implement-if-missing); the category-wide prerequisites are in `global-requirements-data.md`.
- `03-implementation.md` — exact, ordered, click-by-click steps that name and reuse every object (baseline, groups, capacity, workspaces, pipeline).
- `05-testing.md` — validation procedures with pass/fail gates, referencing the scripts by name.
- `06-troubleshooting.md` — symptom → cause → fix for the common failures.

- `07-best-practices.md` — keep-it-clean guidance and where to go next in the catalog.
- `08-deliverables-summary.md` — this file.
- `global-requirements-data.md` — the category-level prerequisites shared by all Data solutions.

Scripts (PowerShell — read-only or preview-first)

- `FABGOV-Discovery.ps1` — read-only platform inventory: tenant settings of interest, capacities, workspaces (orphaned/personal flagged), and a sharing/export summary from the activity log.
- `FABGOV-Validate.ps1` — validates the governance baseline against the intended model: tenant-setting scope, workspace roles (groups only), capacity assignment, and pipeline stage pairing; reports pass/fail per check. Read-only.
- `FABGOV-Report.ps1` — capacity utilization and activity-log summary report (top sharers, exports, publish-to-web attempts) for the weekly governance review. Read-only.
- `FABGOV-TenantSettings-Baseline.ps1` — exports current tenant settings and compares them to a desired-state baseline file, producing a diff; any apply is preview-first and explicitly gated (`-Apply` off by default).
- `FABGOV-Workspace-Provision.ps1` — provisions the `Sales BI – Content – Dev/Test/Prod` workspaces, assigns them to the capacity, and adds group-based roles; runs `-WhatIf` by default and only makes changes when explicitly confirmed.

Naming convention

All objects follow `<Purpose> – <Scope> – <Stage>` and are reused by exact name across the guide, testing, troubleshooting, and scripts (for example `Tenant Governance Baseline – Fabric/Power BI – v1`, `Deployment Pipeline – Sales BI – DevTestProd`).

Format

Delivered as a Complete Solution Package (PDF + Word + scripts) in a single ZIP — `FABGOV-Complete-Solution.zip` — yours to keep and reuse on every rollout.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

FABGOV-Discovery.ps1

Purpose: Captures the "before" picture for the Assess step: the capacities in the tenant, a workspace inventory (flagging personal/orphaned workspaces), and a sharing/export summary from the activity log. Also confirms the admin/scanner APIs are reachable. Everything is read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\FABGOV-Discovery.ps1 Optional: -Days 7 (activity-log look-back window; default 7, max 30 per admin API) Requires the MicrosoftPowerBIMgmt module and the Fabric/Power BI Administrator role (interactive/delegated sign-in). Unattended runs use a service-principal per requirement 2.6. EXPECTED RESULTS Tables of capacities and workspaces, a count of orphaned/personal workspaces, and a top-N summary of share / export / publish-to-web activity — saved to a timestamped CSV set. .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. No tenant-specific values are hard-coded. > [CmdletBinding()] param([ValidateRange(1, 30)] [int]$Days = 7, [string]$OutputFolder = ".\FABGOV-Discovery-Output")

How to run: powershell -ExecutionPolicy Bypass -File .\FABGOV-Discovery.ps1 Optional: -Days 7 (activity-log look-back window; default 7, max 30 per admin API) Requires the MicrosoftPowerBIMgmt module and the Fabric/Power BI Administrator role (interactive/delegated sign-in). Unattended runs use a service-principal per requirement 2.6. EXPECTED RESULTS Tables of capacities and workspaces, a count of orphaned/personal workspaces, and a top-N summary of share / export / publish-to-web activity — saved to a timestamped CSV set. .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. No tenant-specific values are hard-coded. > [CmdletBinding()] param([ValidateRange(1, 30)] [int]$Days = 7, [string]$OutputFolder = ".\FABGOV-Discovery-Output")

FABGOV-Report.ps1

Purpose: Produces the standing governance report: per-capacity metadata (SKU/state/region) and a rolled-up activity summary from the admin activity log — top share/export/publish-to-web actions, top actors, and workspace creation/deletion — so the governance review can spot sprawl and risky trends. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\FABGOV-Report.ps1 -Days 7 Unattended (scheduled) runs use a service principal per requirement 2.6: .\FABGOV-Report.ps1 -ServicePrincipal -TenantId <guid> -AppId <guid> -AppSecret <secret> Requires MicrosoftPowerBIMgmt and read-only admin API access. EXPECTED RESULTS A console summary plus an HTML + CSV report written to the output folder, ready to attach to the weekly governance review. .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. Secrets are parameters, never hard-coded; prefer a managed secret store when scheduling. > [CmdletBinding()] param([ValidateRange(1, 30)][int]$Days = 7, [switch]$ServicePrincipal, [string]$TenantId, [string]$AppId, [string]$AppSecret, [string]$OutputFolder = ".\FABGOV-Report-Output")

How to run: powershell -ExecutionPolicy Bypass -File .\FABGOV-Report.ps1 -Days 7 Unattended (scheduled) runs use a service principal per requirement 2.6: .\FABGOV-Report.ps1 -ServicePrincipal -TenantId <guid> -AppId <guid> -AppSecret <secret> Requires MicrosoftPowerBIMgmt and read-only admin API access. EXPECTED RESULTS A console summary plus an HTML + CSV report written to the output folder, ready to attach to the weekly governance review. .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. Secrets are parameters, never hard-coded; prefer a managed secret store when scheduling. > [CmdletBinding()] param(

[ValidateRange(1, 30)][int]$Days = 7, [switch]$ServicePrincipal, [string]$TenantId, [string]$AppId, [string]$AppSecret, [string]$OutputFolder = ".\FABGOV-Report-Output")

FABGOV-TenantSettings-Baseline.ps1

Purpose: Exports the tenant's current admin (tenant) settings and compares them to a desired-state baseline JSON that represents 'Tenant Governance Baseline - Fabric/Power BI - v1', producing a human-readable drift report (setting, live value, baseline value, scope). By default it is REPORT-ONLY and writes nothing back. -Apply is intentionally gated and, even then, only previews unless -Confirm is added. HOW TO RUN Export a starter baseline from the live tenant: .\FABGOV-TenantSettings-Baseline.ps1 -ExportBaseline .\baseline.json Compare live settings to a reviewed baseline (report-only): .\FABGOV-TenantSettings-Baseline.ps1 -Baseline .\baseline.json (Apply is deliberately restricted — see NOTES. Tenant-setting changes should be made in the admin portal, piloted on 'Governance Pilot Ring - Fabric - Ring1', and change-approved.) Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. EXPECTED RESULTS A drift table of settings that differ from the baseline; an exported baseline file when -ExportBaseline is used. No changes to the tenant in report-only mode. .NOTES Data — Fabric/Power BI Governance. Reads GetTenantSettings via the admin API. Writing tenant settings via API is limited/preview and high-blast-radius; this script therefore treats the admin PORTAL (piloted + change-approved) as the source of change and stays report-first. Verify current admin-API tenant-settings write support in your tenant before enabling any apply.* White-label. No tenant values hard-coded. > [CmdletBinding()] param([string]$Baseline, [string]$ExportBaseline, [switch]$Apply, # gate 1: opt-in to the (preview-only) apply path [switch]$Confirm # gate 2: without this, -Apply still only previews)

How to run: Export a starter baseline from the live tenant: .\FABGOV-TenantSettings-Baseline.ps1 -ExportBaseline .\baseline.json Compare live settings to a reviewed baseline (report-only): .\FABGOV-TenantSettings-Baseline.ps1 -Baseline .\baseline.json (Apply is deliberately restricted — see NOTES. Tenant-setting changes should be made in the admin portal, piloted on 'Governance Pilot Ring - Fabric - Ring1', and change-approved.) Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. EXPECTED RESULTS A drift table of settings that differ from the baseline; an exported baseline file when -ExportBaseline is used. No changes to the tenant in report-only mode. .NOTES Data — Fabric/Power BI Governance. Reads GetTenantSettings via the admin API. Writing tenant settings via API is limited/preview and high-blast-radius; this script therefore treats the admin PORTAL (piloted + change-approved) as the source of change and stays report-first. Verify current admin-API tenant-settings write support in your tenant before enabling any apply.* White-label. No tenant values hard-coded. > [CmdletBinding()] param([string]$Baseline, [string]$ExportBaseline, [switch]$Apply, # gate 1: opt-in to the (preview-only) apply path [switch]$Confirm # gate 2: without this, -Apply still only previews)

FABGOV-Validate.ps1

Purpose: Checks the governance state against the design in the implementation guide and reports pass/fail per check: (1) the lifecycle workspaces are on the expected capacity, (2) workspace roles are group-based only (no individual users), (3) the deployment pipeline stages are paired to the right workspaces. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\FABGOV-Validate.ps1 Optional overrides: -Capacity 'Fabric Capacity - Analytics - Prod' -WorkspacePrefix 'Sales BI - Content' -Pipeline 'Deployment Pipeline - Sales BI - DevTestProd' Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. EXPECTED RESULTS A PASS/FAIL line per check with the offending object named on any failure, and a non-zero exit code if any check fails (so it can gate a pipeline). .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. Object names are parameters; defaults match the implementation guide. >

[CmdletBinding()] param([string]$Capacity = 'Fabric Capacity - Analytics - Prod', [string]$WorkspacePrefix = 'Sales BI - Content', [string]$Pipeline = 'Deployment Pipeline - Sales BI - DevTestProd')

How to run: powershell -ExecutionPolicy Bypass -File .\FABGOV-Validate.ps1 Optional overrides: -Capacity 'Fabric Capacity - Analytics - Prod' -WorkspacePrefix 'Sales BI - Content' -Pipeline 'Deployment Pipeline - Sales BI - DevTestProd' Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. EXPECTED RESULTS A PASS/FAIL line per check with the offending object named on any failure, and a non-zero exit code if any check fails (so it can gate a pipeline). .NOTES Data — Fabric/Power BI Governance. Read-only against the Power BI admin REST APIs. White-label. Object names are parameters; defaults match the implementation guide. > [CmdletBinding()] param([string]$Capacity = 'Fabric Capacity - Analytics - Prod', [string]$WorkspacePrefix = 'Sales BI - Content', [string]$Pipeline = 'Deployment Pipeline - Sales BI - DevTestProd')

FABGOV-Workspace-Provision.ps1

Purpose: Creates the three lifecycle workspaces ('Sales BI - Content - Dev/Test/Prod'), assigns Test and Prod to the governance capacity, and adds the group-based role assignments from the access model. It runs in -WhatIf mode BY DEFAULT (previewing every action); it only makes changes when -Execute is passed. HOW TO RUN Preview (default, safe): .\FABGOV-Workspace-Provision.ps1 Execute for real (after reviewing the preview): .\FABGOV-Workspace-Provision.ps1 -Execute Optional overrides: -WorkspacePrefix 'Sales BI - Content' -Capacity 'Fabric Capacity - Analytics - Prod' Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. The named security groups must already exist (requirement 2.3). EXPECTED RESULTS In preview: a WhatIf line for each workspace create, capacity assignment, and role add. With -Execute: the workspaces created/assigned/roled, then a re-run of the model summary. .NOTES Data — Fabric/Power BI Governance. Change script — preview-first, explicitly gated by -Execute. White-label. Group and object names are parameters; defaults match the implementation guide. > [CmdletBinding()] param([switch]$Execute, [string]$WorkspacePrefix = 'Sales BI - Content', [string]$Capacity = 'Fabric Capacity - Analytics - Prod', Access model: stage -> @{ Group role } (roles: Admin, Member, Contributor, Viewer) [hashtable]$Roles = @{ 'Dev' = @{ 'BI Creators - Analytics - Pilot' = 'Contributor'; 'Fabric Admins - Platform - Prod' = 'Admin' } 'Test' = @{ 'BI Content Certifiers - Analytics - Prod' = 'Member'; 'Fabric Admins - Platform - Prod' = 'Admin' } 'Prod' = @{ 'Fabric Admins - Platform - Prod' = 'Admin' } })

How to run: Preview (default, safe): .\FABGOV-Workspace-Provision.ps1 Execute for real (after reviewing the preview): .\FABGOV-Workspace-Provision.ps1 -Execute Optional overrides: -WorkspacePrefix 'Sales BI - Content' -Capacity 'Fabric Capacity - Analytics - Prod' Requires MicrosoftPowerBIMgmt and the Fabric/Power BI Administrator role. The named security groups must already exist (requirement 2.3). EXPECTED RESULTS In preview: a WhatIf line for each workspace create, capacity assignment, and role add. With -Execute: the workspaces created/assigned/roled, then a re-run of the model summary. .NOTES Data — Fabric/Power BI Governance. Change script — preview-first, explicitly gated by -Execute. White-label. Group and object names are parameters; defaults match the implementation guide. > [CmdletBinding()] param([switch]$Execute, [string]$WorkspacePrefix = 'Sales BI - Content', [string]$Capacity = 'Fabric Capacity - Analytics - Prod', Access model: stage -> @{ Group role } (roles: Admin, Member, Contributor, Viewer) [hashtable]$Roles = @{ 'Dev' = @{ 'BI Creators - Analytics - Pilot' = 'Contributor'; 'Fabric Admins - Platform - Prod' = 'Admin' } 'Test' = @{ 'BI Content Certifiers - Analytics - Prod' = 'Member'; 'Fabric Admins - Platform - Prod' = 'Admin' } 'Prod' = @{ 'Fabric Admins - Platform - Prod' = 'Admin' } })